Python-Based Simulator for Dynamic Workload Adaptation and Performance Analysis in Classical CPU Scheduling Algorithms

Joga Eswar Inish
Department of Electronics and Communication Engineering
Amrita Vishwa Vidyapeetham
Kollam, Kerala, India
am.en.u4ece23118@am.students.amrita.edu

Kalimuthu Vishnu Varadhan
Department of Electronics and Communication Engineering
Amrita Vishwa Vidyapeetham
Kollam, Kerala, India
am.en.u4ece23119@am.students.amrita.edu

Ragesh Kannan Megalingam
Department of Electronics and Communication Engineering
Amrita Vishwa Vidyapeetham
Kollam, Kerala, India

***Abstract*—CPU scheduling is an important function of an operating system. It influences how responsive computing systems are, as well as their throughput, turnaround time, and waiting time. Common scheduling methods include First Come First Served (FCFS), Shortest Job First (SJF), Priority Scheduling, and Round Robin (RR). They come with their own advantages and disadvantages, and these algorithms are evaluated under synthetic workloads. This project focuses on implementing four conventional algorithms using a Dynamic Workload Adaptation module, which is implemented using a Python-based simulator. The module continuously monitors workload characteristics—such as arrival patterns, burst times, and process types—and adjusts scheduler parameters, including RR quantum and priority ageing, during execution. The goal is twofold: (1) Comparison of how each algorithm performs across the synthetic workload mixes (interactive, CPU-bound, I/O-bound, and bursty arrivals) and (2) Estimate if any basic adaptive rules can improve fairness and latency without affecting the throughput. This module will measure the turnaround time, waiting time, response time and context-switch overhead across scenarios. The module's outcome demonstrates how minor adjustments can improve the performance or cause an extra overhead, finding out which algorithm works best in different situations.**

***Index Terms*—CPU Scheduling, Operating Systems, Round Robin, Dynamic Workload, Python Simulation**

I. Introduction

Efficient CPU (Central Processing Unit) scheduling has been one of the most important and challenging problems in computer science and operating systems design. The CPU, Central Processing Unit, is the most important part. Several processes compete for execution simultaneously. Hence, it becomes essential to decide which process runs first and in what sequence. This emphasises the significance of scheduling. Efficient CPU scheduling is an important factor in deciding the turnaround time, waiting time, response time and overall throughput.

From batch processing to real-time environments, computing systems have come a long way. With it, the need for better scheduling also increases. Earlier batch systems focused on maximising throughput whilst also minimising idle time on CPU; however, with the rise of real-time and embedded systems, this focus turned to providing predictability and guaranteeing deadlines. Modern cloud and distributed platforms focus on scalability, adaptability, and efficient use of resources. Despite the evolution, the four popular scheduling algorithms (First Come First Served (FCFS), Shortest Job First (SJF), Priority Scheduling, and Round Robin (RR)) remained baseline methods of scheduling. Each scheduling algorithm has a different perspective as to how scheduling should happen. First Come First Served (FCFS) is simple in idea, but suffers from the convoy (where one or more long-duration processes hold the CPU, causing all the shorter processes to wait in the queue). Shortest Job First (SJF) tries to reduce the average waiting time but requires an accurate prediction of the burst time, which is difficult in real-time systems. Priority Scheduling differentiates service by assigning a priority level to each process, but has a risk of starvation (a low-priority process is unable to get the CPU time it needs, even though it is ready to run due to high-priority processes). Round Robin (RR) attempts to maintain a fixed time quantum (the predetermined amount of time a process is allowed to run on a CPU before being sent to the round robin algorithm) to improve responsiveness, while frequent context switching affects its performance. From this, none of these algorithms is universally optimal; these algorithms have their individual advantages and disadvantages, which depend on the nature of the workload.

This project builds a Dynamic Workload Adaptation module which implements the four conventional algorithms using a Python-based simulator. The aim is to compare algorithm performance across different synthetic workload mixes, including interactive, CPU-bound, I/O-bound, and bursty arrivals. The module allows for minor adjustments, such as modifying

the time quantum in the Round Robin (RR) algorithm or adjusting priority levels based on workload conditions. It also explores how minor adaptations can improve performance without compromising throughput.

II. Related Works/Literature Review

CPU scheduling is the main reason for any computer system running smoothly, it doesn't matter if it's a simple desktop or a massive cloud setup. The classic algorithms like First Come First Serve (FCFS), Shortest Job First (SJF), Priority scheduling, and Round Robin (RR) have been studied over for years. For example, Reddy and their team [6] tested these under different workloads and showed how they compare in terms of response time, waiting time, and turnaround time. Marphatia's team [5] improved the First Come First Serve (FCFS) method for cloud systems by using smarter ways to assign resources. They showed that adjusting the schedule to fit the workload can really speed things up. And for Round Robin (RR), with approaches to select the time quantum adaptively [7] and [10] significantly decreasing context-switching overhead and increasing fairness.

The other set of studies emphasises scheduling for real-time and priority systems. Thekkilakattil and others [2] used CPU frequency scaling to avoid too many interruptions, resulting in improved predictability and stability in real time systems. This gave us the idea for how our simulator handles priorities when there's a sudden increase in workload. Xu's team [8] created priority schedulers that adjust based on changing needs, and that idea helps us make real-time changes in our simulator. Cormen and Katz [9] showed how to schedule tasks on multiple processors without missing deadlines—really helpful for handling lots of tasks at once. Fewer interruptions and smarter priority handling make systems more reliable and faster.

To test scheduling fairly, you need solid workload models and ways to create realistic tasks. Sindhu's work [1] used random methods to simulate how cloud traffic behaves, so researchers could test their algorithms in many different situations. Sharma and Gupta [12] studied how CPU scheduling works when cloud traffic keeps changing. Singh and Gill [13] combined traditional methods with AI to make smarter, more flexible systems. These ideas helped our simulator handle different types of tasks—like interactive ones, CPU-heavy jobs, I/O-heavy work, or sudden spikes—so we can really see how well the algorithms perform.

Resource efficiency, especially energy and load balancing, has grown in importance in modern computing. S. V. and team [4] showed how task scheduling in cloud-fog systems can cut down power use. Gupta and Kumar [15] added load balancing mechanisms that saves energy, and Chen's team [11] worked on meeting deadlines without wasting resources. Scheduling is no longer only concerned with fairness or throughput but also with sustainability and efficiency in resource-constrained environments.

Modern studies are shifting towards the use of optimisation methods and intelligent algorithms. Singh and others [3] compared Genetic Algorithms with Flower Pollination Algorithms for cloud computing tasks and showed that these smart techniques can make basic methods work much better. Li's study [14] worked on finding the right balance between cost, energy use, and speed all at once. And Das and team [16] created AI systems that automatically adjust settings as needed. It marks a change from conventional algorithms to clever, flexible algorithms that improve performance even in uncertain circumstances.

III. DESIGN AND IMPLEMENTATION

A. Block Diagram

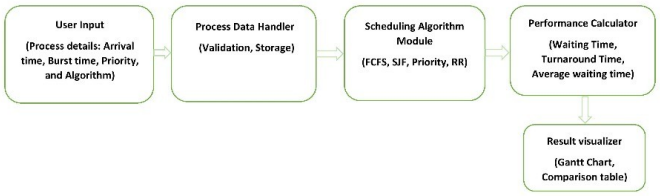


Fig. 1. Block diagram of the CPU scheduling algorithm.

The block diagram (Fig. 1) gives us the complete overview of the CPU Scheduling Algorithm Simulator, with the main functional modules and data flow. It begins with user input, where parameters such as arrival time, burst time, priority, algorithm choice, and time quantum (if RR) are given to the simulator. The given data will be processed using the process data handler, which validates and stores input as arrays or lists. Post the data validation, it goes to the scheduling algorithm's part, in which the four traditional algorithms (FCFS, SJF, Priority, and RR) compute the process execution. Then the performance gets calculated, which calculates the parameters – waiting time, turnaround time, average waiting time, and average turnaround time. These results are sent to the result visualizer, where the Gantt chart and a table are used to compare the performance of scheduling algorithms. The block diagram visualizes the simulator from data input to output visualization, which gives us an idea of the system's operation and data processing flow.

B. Flowchart/Algorithm

The flowchart (Fig. 2) gives a detailed step-by-step process explanation of the CPU Scheduling Algorithm Simulator. It starts with initializing the program and asking the user to enter the required parameters – number of processes, burst time, arrival time, and priority. After these parameters are given to the simulator, the user selects the algorithm. The simulator arranges the processes based on the selected algorithm. For example, First Come First Serve (FCFS) sorts by arrival order, Shortest Job First (SJF) selects the smallest burst time first, Priority executes based on the rank, and Round Robin (RR) uses time slicing- in which a fixed amount of processing

time is allocated to each task. After the scheduling part, the algorithm calculates the individual and average values of waiting time and turnaround time, and other metrics to measure efficiency.

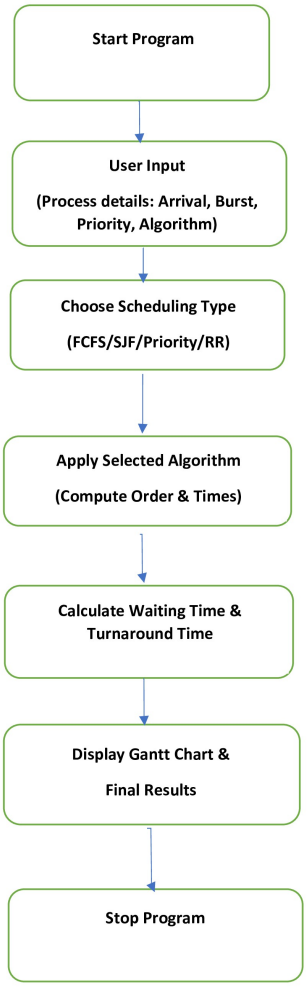


Fig. 2. Flowchart of the CPU scheduling algorithm

C. Implementation/Pseudo Code

Algorithm 1 Process Initialization

```
class Process
function INITIALIZE(pid, arrival, burst, priority = 0)
    self.pid ← pid
    self.arrival ← arrival
    self.burst ← burst
    self.remaining ← burst
    self.priority ← priority
    self.completion ← 0
    self.waiting ← 0
    self.turnaround ← 0
end function
```

Algorithm 2 FCFS Scheduling Algorithm

```
function FCFS_SCHEDULING(process_list, start_time)
    Sort process_list by arrival time
    time ← start_time for each process in process_list do
_
     time < process.arrival
    time ← process.arrival

    Execute process from time to time + burst
    Update completion, waiting, turnaround times
    time ← time + process.burst

    return gantt_chart, time
end function
```

Algorithm 3 Shortest Job First (SJF) Scheduling Algorithm

```
function SJF_SCHEDULING(process_list, start_time)
    Sort process_list by arrival time while not all processes
_completed do
     Select process with minimum burst time among arrived
    Execute process
    Update completion, waiting, turnaround times

    return gantt_chart, time
end function
```

Algorithm 4 Round Robin Scheduling Algorithm

```
function ROUND_ROBIN(process_list, quantum, start_time)
    Sort process_list by arrival time
    Initialize ready_queue while ready_queue not empty or
_processes remaining do
     Add arrived processes to queue
    Select first process
    Execute for min(remaining, quantum)
    Update remaining time and system time if process
_incomplete then
_     Reinsert into queue else
_     Update completion, waiting, turnaround

    return gantt_chart, time
end function
```

Algorithm 5 Priority Scheduling Algorithm

```
function PRIORITY_SCHEDULING(process_list, start_time)
    Sort process_list by arrival time while not all processes
_completed do
     Select process with highest priority among arrived
    Execute process
    Update completion, waiting, turnaround

    return gantt_chart, time
end function
```

Algorithm 6 Performance Table Generation

function PERFORMANCE_TABLE(process_list) **for** *each process in process_list* **do**

Calculate waiting and turnaround times

Compute average waiting time and average turnaround time

Display results in tabular format

=0

Algorithm 7 Gantt Chart Plotting

function PLOT_GANTT(gantt_chart, title) **for** *each (pid, start, end) in gantt_chart* **do**

Draw horizontal bar for process duration

Label axes and add title

Algorithm 8 Dynamic Scheduler with Workload Adaptation

function DYNAMIC_SCHEDULER(process_list, quantum)

Print "Simulation with Dynamic Workload Adaptation started"

Divide process_list into:

initial_batch $\leftarrow$ processes with arrival ≤ 3

later_batch $\leftarrow$ processes with arrival > 3

Run **SJF_Scheduling** on initial_batch **if** *new processes arrive* **then**

Run **Round_Robin** on later_batch with quantum

Combine Gantt charts and compute performance metrics

Visualize final results

Print "Simulation completed"

end function=0

Algorithm 9 Main Program Execution

```
Read number_of_processes from user for each process do
    Input arrival_time, burst_time, priority
Create process object and add to list

Call Dynamic_Scheduler(process_list)
```

The pseudo-code shows the complete workflow and the operation process of the CPU scheduling algorithms with dynamic workload adaptation. It consists of all the necessary process attributes: arrival time, burst time, priority, and waiting time, turnaround time and completion time, which are changed during the process, and these attributes are called updated values. The simulator executes the four scheduling algorithms FCFS, SJF, Priority Scheduling, and Round Robin (RR). All the scheduling algorithms have a separate function for each one to be simulated. The execution method of FCFS schedules the operations according to their arrival times and makes no provision for the current process to be interrupted. The SJF method of operation is the one that minimizes the average waiting time by selecting the process with the least burst duration among those that have arrived. The Priority Scheduling technique grants the CPU to the process that has the utmost priority rating, and the Round Robin technique divides the CPU time into equal time intervals (quantum), giving every job the same amount of time. After this, the performance table generation module computes and shows the average waiting time and turnaround time. The Gantt chart plotting algorithm is used to visualise the CPU execution timeline, clearly showing how each process utilises the CPU time. The dynamic scheduler algorithm divides incoming processes into two categories- initial and later batches, depending upon the arrival time. The initial batch is executed using SJF to optimise short tasks, while later batches are scheduled using Round Robin to ensure fairness and responsiveness for new arrivals. The simulation of real-time adaptation is done by this process; the scheduler has to make a dynamic choice of the algorithm depending on the system load and the behaviour of the arriving processes. All these modules are integrated by the main program, and all the necessary input data is gathered from the user. After which, the dynamic scheduler is executed, and the performance table and Gantt chart are used to display the results.

D. Overall Working

The simulator starts with the execution of an input module, where the user enters the parameters, such as arrival time, burst time, priority, algorithm choice, and time quantum if Round Robin is selected; all these are given to the simulator. After the giving the input, the user should select any one of the algorithms. The algorithm arranges the processes according to the selected algorithm logic, and a queue is maintained throughout, making sure each process is allocated to the CPU correctly. During the execution, the algorithm gets updated with the completion, waiting, and turnaround times, simulating a real CPU environment. The simulator acts like a real CPU scheduler by recording and reporting completion time, waiting time, and turnaround time for each process during the simulation. By using this simulator to store and constantly update these values, it is possible to witness the impact of scheduling on the CPU's performance and efficiency directly, providing an environment similar to the real one for the analysis of the scheduling algorithms.

After completion of this process, this simulator starts working on the performance analysis part. The average waiting time, average turnaround time, and overall throughput are calculated. The simulator then plots a Gantt chart. The simulator additionally displays the total period of execution. along with this, the simulator also features a table for performance comparison that sums up the outcomes for all four algorithms– First Come First Serve, Shortest Job First , Priority, and Round Robin. It is under the same workload conditions that the table shows the pros and cons of each method. The simulator shows

how traditional CPU scheduling algorithms can be analysed using the simulator. This working simulates a complete process life cycle from user input to performance visualisation. Using both Python code and a Gantt chart, it provides a clear and practical visual understanding of how the CPU scheduling algorithms affect the efficiency of the system. This simulator can be used for any extendable framework for studying and analysing classical CPU scheduling in Python.

IV. Experiments and Results

The simulator was implemented in Google Colab with Python. *Matplotlib* and *Pandas* were used for tabulation and data visualization. Four parameters—*Process ID*, *Arrival Time*, *Burst Time*, and *Priority*—were used to visualize the entire process as an object. During execution, the parameters *Waiting Time*, *Turnaround Time*, and *Completion Time* were dynamically updated. The selected algorithm from *First-Come-First-Served (FCFS)*, *Shortest Job First (SJF)*, *Round Robin (RR)*, and *Priority Scheduling (PS)* was executed by the simulator using the given process parameters. The integration of the *Dynamic Workload Adaptation* module ensured that processes arriving before time 3 were executed using the *Shortest Job First (SJF)* algorithm, while those arriving afterward were executed using the *Round Robin (RR)* algorithm with a time quantum of 2. The simulator dynamically adjusted the scheduling method based on the workload. Each algorithm generated a *Gantt chart* showing the process execution order, a *performance table* displaying the waiting and turnaround times, and a *comparative bar graph* for overall efficiency analysis.

A. Results

1) First Come First Serve (FCFS) Scheduling: The First Come First Serve (FCFS) algorithm executed processes in the order of arrival. The Gantt chart (Fig. 3) indicated that later-arriving processes experienced higher waiting times due to the sequential nature of execution.

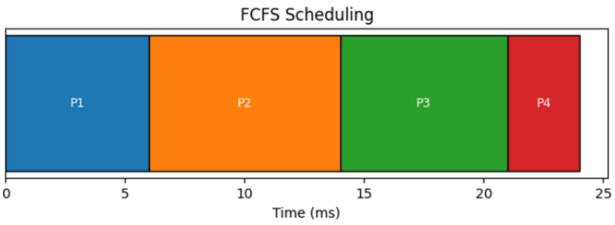


Fig. 3. Gantt Chart – FCFS Scheduling

First Come First Serve (FCFS) is simple but suffers from the convoy effect, where long processes delay shorter ones. It is best suited for smaller workloads but inefficient for time-sharing systems.

2) Shortest Job First (SJF) Scheduling: The Shortest Job First (SJF) algorithm executes processes with the shortest burst times first. The Gantt chart (Fig. 4) shows that shorter jobs were prioritized, reducing average waiting time.

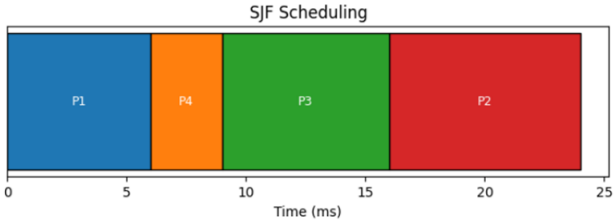


Fig. 4. Gantt Chart – SJF Scheduling

The Shortest Job First (SJF) method achieved the lowest average waiting time of 6.25 ms and high throughput. However, starvation can occur when shorter processes continuously arrive, delaying longer ones.

3) Round Robin (RR) Scheduling: The Round Robin (RR) algorithm was simulated with a time quantum of 2 units. From the Gantt chart (Fig. 5) it is evident that frequent context switches ensured fairness but slightly increased turnaround time.

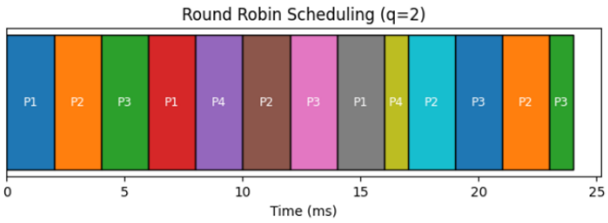


Fig. 5. Gantt Chart – RR Scheduling

The Round Robin (RR) method is ideal for interactive and real-time systems. Large quantum values mimic First Come First Serve (FCFS) behavior, while very small values cause high switching overhead.

4) Priority Scheduling: In Priority Scheduling, each process is assigned a priority level, with smaller numbers indicating higher importance. High-priority processes were executed first and completed their burst time before lower-priority ones.

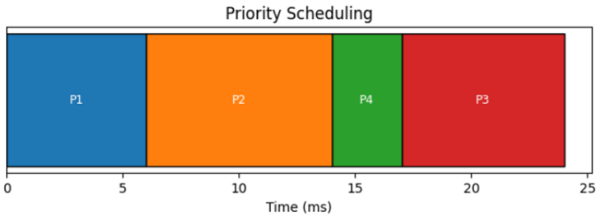


Fig. 6. Gantt Chart – Priority Scheduling

From the Gantt chart (Fig. 6), this method performs effectively when priority levels are appropriately assigned. However, it can lead to starvation for low-priority processes.

5) Dynamic Workload Adaptation: In the final assessment, the system employed Shortest Job First (SJF) for processes arriving before time 3 and switched to Round Robin (RR) for those arriving afterward. This simulated dynamic scheduling adaptation.

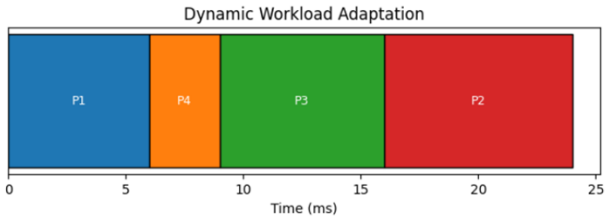


Fig. 7. Gantt Chart – Dynamic Workload Adaptation

The Gantt chart (Fig. 7) shows that the early processes P1, P4, and P3 are executed in sequence using Shortest Job First (SJF), and the other process, P2, is executed using Round Robin (RR) algorithm to ensure fair CPU allocation of time. The hybrid scheduler achieved fairness and reduced waiting time by combining Shortest Job First (SJF) efficiency with Round Robin (RR) fairness, resulting in balanced performance under varying loads.

B. Performance Comparison

Table I shows the performance metrics for each scheduling algorithm based on average waiting and turnaround times.

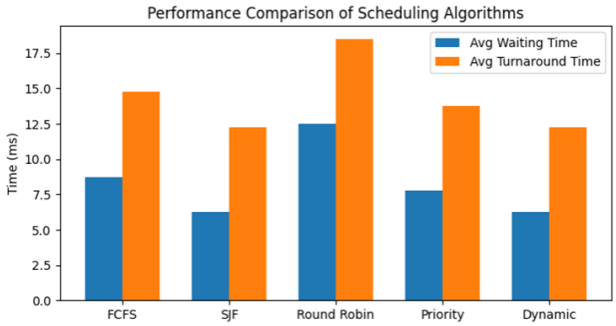


Fig. 8. Performance Comparison of Scheduling Algorithms

TABLE I
COMPARATIVE PERFORMANCE OF SCHEDULING ALGORITHMS

Algorithm	Average Waiting Time (ms)	Average Turnaround Time (ms)
First Come First Serve (FCFS)	8.75	14.75
Shortest Job First (SJF)	6.25	12.25
Round Robin (RR)	12.50	18.50
Priority Scheduling	7.75	13.75
Dynamic Workload Adaptation	6.25	12.25

From (Fig .8) and Table 1, Shortest Job First (SJF) performed best under static workloads, achieving the lowest waiting time and high throughput. The Dynamic Workload Adaptation approach maintained performance consistency across varying loads, balancing fairness and responsiveness.

First Come First Serve (FCFS) and Round Robin (RR) exhibited higher latencies due to sequential and time-sliced execution, respectively. Priority Scheduling showed efficiency only when priorities were assigned accurately.

C. Overall Observation

The following conclusions were derived from the experiments:

First Come First Serve (FCFS): Simple and non-preemptive, but inefficient for mixed workloads.
Shortest Job First (SJF): Provides optimal performance for short tasks but suffers from starvation.
Round Robin (RR): Ensures fairness but incurs context-switching overhead.
Priority Scheduling: Effective for critical systems but prone to starvation.
Dynamic Workload Adaptation: Combines the benefits of SJF and RR, adapting efficiently to changing workloads.

Overall, the simulator successfully demonstrated the behavior of classical CPU scheduling algorithms. The Gantt charts and performance graphs generated using Python clearly show that the adaptive model improves fairness and reduces average waiting time without compromising throughput.

V. CONCLUSION

This project successfully implements a Python-based simulator that analyses the performance of traditional CPU scheduling algorithms, and it has a dynamic workload adaptation module, which is a combination of Shortest Job First (SJF) for early arriving processes and Round Robin (RR) for later arriving processes, which leads to improved fairness and responsiveness. The results and the Gantt charts show the working of each algorithm, showing how scheduling decisions affect the waiting time, turnaround time, and overall throughput. The dynamic workload adaptation module has achieved a balance between efficiency and fairness, showing the behavior of modern operating systems. Overall, the simulator gives an understanding of traditional CPU scheduling algorithms.

VI. FUTURE WORKS

The Python-based simulator is based on non-preemptive scheduling and a basic dynamic workload adaptation module; it can be improved in many ways. Preemptive scheduling variants of Shortest Job First (SJF) and Priority Scheduling can be added to model real-time operating systems more accurately. Machine Learning techniques can be used to improve the dynamic workload adaptation module by automatically choosing scheduling policies based on workload patterns or predicting optimal time quantum values. The simulator's functionality and practicality can be further improved by support for I/O-bound processes, multi-CPU/core scheduling, and Gantt chart export options. The simulator can become a comprehensive framework for CPU scheduling analysis.

REFERENCES

[1] K. Sindhu, K. Seshadri, and C. Kollengode, "Workload characterization and synthesis for cloud using generative stochastic processes," J. Supercomput., vol. 78, pp. 18825–18855, 2022.
[2] A. Thekkilakattil, A. S. Pillai, R. Dobrin, and S. Punnekkat, "Reducing the number of preemptions in real-time systems scheduling by CPU frequency scaling," Proc. RTNS, 2010.
[3] P. Singh, G. Bathla, D. Panwar, A. Aggarwal, and S. Gaba, "Performance evaluation of genetic algorithm and flower pollination algorithm for scheduling tasks in cloud computing," Springer, 2023.
[4] P. M. S. V. and M. K. P., "Energy-efficient task scheduling and resource allocation for cloud–fog environment," Symmetry, 2022.

[5] A. Marphatia et al., “Optimization of FCFS based resource provisioning algorithm for cloud computing,” IOSR-JCE, 2013.
[6] M. R. Reddy et al., “Comparative analysis of CPU scheduling algorithms,” Proc. ICCMC, 2019.
[7] K. R. Ramesh Babu and A. V. Reddy, “Dynamic Round Robin CPU scheduling algorithm,” Proc. ICACCI, 2017.
[8] J. Xu, Z. Chen, and L. Chen, “Priority-based task scheduling in cloud computing,” Proc. CloudCom, 2015.
[9] T. H. Cormen and J. Katz, “Efficient scheduling algorithms for real-time multiprocessor systems,” Proc. IEEE RTSS, 2016.
[10] R. Kaur and S. Kinger, “Enhanced Round Robin scheduling using dynamic time quantum,” Proc. INDIACom, 2014.
[11] Y. Chen et al., “Deadline-aware task scheduling in cloud systems,” Proc. IEEE ICPADS, 2018.
[12] A. Sharma and V. Gupta, “Comparative study of CPU scheduling algorithms,” Proc. ICIT, 2017.
[13] H. Singh and R. Gill, “Hybrid approach to CPU scheduling,” Proc. ICCSE, 2020.
[14] J. Li, H. Zhang, and Y. Wang, “Multi-objective optimization for task scheduling in cloud computing,” Future Gener. Comput. Syst., 2020.
[15] N. Gupta and M. Kumar, “Load balancing and energy-aware scheduling in cloud computing,” J. Cloud Comput., 2020.
[16] R. Das, S. Misra, and V. Bhatt, “AI-driven optimization techniques for CPU scheduling,” IEEE Access, 2020.